Traffic-System: A Next.js-Based Smart Traffic Enforcement Platform

SoftwareX-Style Seminar Paper Report

Prepared for academic documentation and system presentation

Generated from the current Traffic-System codebase

Workspace:	Traffic-System
Platform:	Next.js, React, MongoDB, FastAPI, PyTorch
Scope:	Citizen services, administrative enforcement, ALPR integration
Document type:	Detailed seminar paper / technical report
Date:	April 22, 2026

Traffic-System: An Integrated Next.js Web Platform with ALPR-Enabled Enforcement Automation for Smart-City Traffic Administration

Abstract

Traffic-System is a full-stack smart-city web platform that unifies citizen vehicle services, administrative enforcement tools, and automated traffic violation workflows in a single browser-based environment. The application is implemented with Next.js and React on the frontend, MongoDB and Mongoose for persistent data management, and a Python-based ALPR service for number plate extraction.

The system is designed to approximate the behavior of a digital traffic authority. Citizens can register vehicles, renew registration, update details, report theft, inspect violation history, and track payment status. Administrators can manage camera nodes, inspect traffic records, simulate violations, visualize camera networks, and run machine-learning plate prediction tests.

A distinguishing aspect of the project is layered ALPR integration. The repository supports both a standalone FastAPI inference server and a local Python fallback path. Plate normalization also handles regional formatting details such as Bengali digits and spacing differences.

Keywords: automatic license plate recognition, traffic enforcement, Next.js, MongoDB, FastAPI, smart city, vehicle registration, violation management

1. Introduction

Traffic enforcement systems involve high operational complexity, continuous data ingestion, and both manual and automated decision paths. Traffic-System addresses this by unifying registry operations, enforcement actions, notifications, and payments in one coherent platform.

The architecture separates citizen and administrator workflows while keeping a shared data model. This enables end-to-end traceability from event detection to user notification and payment status updates.

2. System Architecture

The platform has three interacting layers:

1. Next.js frontend pages for citizen and admin interfaces.

2. Next.js API routes for orchestration, authentication, validation, and business logic.

3. Data and inference layer with MongoDB/Mongoose and a Python ALPR service.

This design keeps web workflows cohesive while allowing specialized machine-learning execution in Python.

3. Complete Feature Coverage

The project includes all major feature groups listed below.

3.1 Citizen Services

- Transfer vehicle ownership.
- Renew vehicle registration.
- Update vehicle and owner information.
- Report stolen vehicles.
- Check registration and enforcement status.
- Review payment history.
- Track tickets, payment state, and driving credit score.
- Track stolen vehicle status and sightings.

3.2 Traffic Monitoring and Enforcement

- Live camera location map.
- Traffic and violation record search.

- Violation management and inspection.
- Accident and traffic news feed publication.
- Violation simulation for admin testing.
- ALPR-based plate extraction and normalization.

3.3 Administrative Tools

- Vehicle registry management.
- Violation record management.
- Traffic analytics dashboard.
- Camera node and network-edge management.
- Newsfeed and carousel content management.
- ML prediction testing sandbox.

4. Frontend Page Inventory (Complete)

Route	Role
/	Landing entry point
/home/landing	Public landing view
/authentication/signin	Sign-in flow
/authentication/signup/vehicle	Sign-up with vehicle registration
/check-email	Verification follow-up
/dashboard	Authenticated dashboard
/profile	User profile
/myvehicle	Vehicle detail screen
/explore	Public exploration and information
/reports	Reporting and analytics page

Route	Role
/tickets&violations	Violation history with map context
/tickets&violation/payment/ticket/[ticketId]	Ticket payment detail page
/services/change-ownership	Ownership transfer
/services/check-status	Status lookup
/services/payment-history	Payment history
/services/renew-registration	Registration renewal
/services/report-stolen	Stolen vehicle reporting
/services/update-details	Vehicle detail updates
/admin	Admin dashboard
/admin/addCamera	Add camera node
/admin/analysis	Analytics and trends
/admin/ml-predict	ALPR prediction test page
/admin/simulate-violation	Violation simulation
/admin/traffic-records	Traffic records browser
/admin/UI	Admin content control area
/admin/UI/carousel	Carousel manager
/admin/UI/newsfeed	Newsfeed manager
/admin/vehicles	Vehicle administration
/admin/violations	Violation administration

5. API Route Inventory (Complete)

Route	Method	Responsibility
/api/authentication/signin	POST	Credential validation and JWT issuance
/api/authentication/signup	POST	Account creation and registration checks
/api/authentication/verifyemail	POST	Email verification flow
/api/violations	GET	User-scoped violation history

Route	Method	Responsibility
/api/vehicles	GET	Vehicle registry listing
/api/traffic-records	GET	Traffic detection and violation records
/api/dashboard/stats	GET	Dashboard summary metrics
/api/dashboard/carousel	GET	Carousel content feed
/api/dashboard/carousel-by-category	GET	Category-filtered carousel feed
/api/dashboard/newsfeed	GET	Traffic and accident newsfeed
/api/notifications	GET	Notification retrieval
/api/notifications/stream	GET	Notification stream endpoint
/api/profile	GET	User profile and linked records
/api/payments/gateway	POST	Payment recording and violation status update
/api/admin/camera	GET/POST/DELETE	Camera node management
/api/admin/camera/[id]	PUT/DELETE	Single camera update/delete
/api/admin/locations	GET	Location listing for map/graph
/api/admin/edges	GET/POST	Road-network edge management
/api/admin/analysis	GET	Traffic trend aggregation
/api/admin/violations/simulate	POST	Synthetic violation generation
/api/admin/violations/extract-plate	POST	ALPR extraction via FastAPI or fallback
/api/admin/UI/newsfeed	GET/POST	Admin newsfeed CRUD operations
/api/admin/UI/carousel/add	POST	Add carousel item
/api/admin/UI/carousel/delete/[id]	DELETE	Delete carousel item
/api/services/add-vehicle	POST	Add citizen vehicle record
/api/services/change-ownership	POST	Ownership transfer workflow
/api/services/check-status	GET	Registration/compliance status check
/api/services/renew-registration	POST	Registration renewal update
/api/services/report-stolen	POST	Stolen flag propagation

Route	Method	Responsibility
/api/services/update-details	POST	Vehicle detail updates

6. Data Models (Complete)

Model	Primary Purpose	Representative Fields
User	Authentication and account identity	email, password hash, role, verification, credit score
Vehicle	Registry and ownership	number plate, owner, status, registration dates
TrafficRecord	Violation/detection events	vehicle, plate, location, timestamp, fine, status
Notification	User alerts	message, read state, linked violation
Location	Monitoring node	name, latitude, longitude
Edge	Weighted graph edge	from, to, distance, travel time, direction
Carousel	Dashboard/public content	image, title, description, category
NewsFeed	Traffic information entries	headline, content, timestamp, category

7. Machine Learning Integration

The ALPR subsystem is implemented in Python and integrates with the Next.js admin workflow. The backend supports two execution modes:

- FastAPI server mode for persistent model loading and lower repeated startup cost.
- Local fallback mode for direct subprocess invocation during development/testing.

Core artifacts include:

- Hybrid Pipeline/hybrid$_pipeline.py$
- scripts/extract$_plate_pipeline.py$
- ml$_service/main.py$
- Hybrid Pipeline/models/*.pth and yolo_plate.pt

Operational endpoints include `GET /health` and `POST /predict` on the FastAPI service.

8. Security, Reliability, and Limitations

The platform uses JWT-based authentication and role-separated route families. Current implementation strengths include clear route boundaries and modular service separation, while limitations include prototype-level payment integration, localStorage token trade-offs, and expected ALPR uncertainty under difficult image conditions.

9. Conclusion

Traffic-System provides a complete applied prototype of smart-city traffic administration with citizen services, administrative enforcement, and ALPR integration. The software is suitable for SoftwareX-style academic reporting because it is modular, reproducible, and broad in operational coverage.

A. Formatting Compliance Note

This LaTeX document follows the same report style goals used in the prior template: title page, abstract, structured technical sections, explicit feature inventories, and appendices. It uses 12pt Times-style typography, 1-inch margins, and double spacing to match the original seminar-report formatting intent.

B. Environment Variables Summary

- DATABASE_URL / MONGO_URL

- NEXTAUTH_SECRET, NEXTAUTH_URL
- ALPR_USE_FASTAPI, ALPR_FASTAPI_URL
- ALPR_MODELS_DIR, ALPR_CITY_LABEL_FILE, ALPR_CHAR_LABEL_FILE
- KMP_DUPLICATE_LIB_OK